# Enhancing Generalization in Large-Scale HCVRP: A Rank-Augmented Neural Solver

Qidong Liu
School of Computer Science and Artificial Intelligence
Zhengzhou University
Zhengzhou, China
ieqdliu@zzu.edu.cn

Jiurui Lian
School of Computer Science and Artificial Intelligence
Zhengzhou University
Zhengzhou, China
ljr314159@gs.zzu.edu.cn

Chaoyue Liu*
School of Computer Science and Artificial Intelligence
Zhengzhou University
Zhengzhou, China
chaoyueliu@gs.zzu.edu.cn

Zhiguang Cao
School of Computing and Information Systems
Singapore Management University
Singapore, Singapore
zgcao@smu.edu.sg

## Abstract

The Heterogeneous Capacitated Vehicle Routing Problem (HCVRP) is an NP-hard combinatorial optimization problem. State-of-the-art neural solvers face difficulties in generalizing to large-scale scenarios after training on small-scale instances. Our experiments reveal that performance degradation is primarily due to the low-rank nature of attention matrix in large-scale instances. This results in insufficient distinction among node features, impacting the accuracy of Markov Decision Processes. Additionally, these models utilize self-attention for vehicle information interaction, but overly incorporate features from others, which suppresses individual features and leads to a deviation from the optimal route. To address these challenges, we propose the Rank-Augmented Neural Solver (RANS), which introduces two key innovations: 1) A simple yet effective mechanism to increase and approximate the upper bound of the attention matrix's rank, enabling the generation of more distinctive node features. 2) A Dual Cross-Attention Module within the vehicle encoder that accurately captures each vehicle's optimal routes while maintaining balanced vehicle collaboration. The experimental results show that RANS performs favorably against the baselines. Notably, when applied to instances with up to 10,000 nodes, RANS achieves an inference time that is merely 13.42% of the best baseline among the neural solvers, while simultaneously reducing the min-max travel time by 23.72%.

## CCS Concepts

• **Computing methodologies** → **Machine learning**; **Markov decision processes**; **Neural networks**; **Learning latent representations**.

*Corresponding author.


*KDD '25, Toronto, ON, Canada*

ACM ISBN 979-8-4007-1454-2/2025/08
https://doi.org/10.1145/3711896.3736935

## Keywords

Combinatorial Optimization; Reinforcement Learning; Attention Mechanism; Rank-Augmented; Heterogeneous CVRP

**ACM Reference Format:**
Qidong Liu, Jiurui Lian, Chaoyue Liu, and Zhiguang Cao. 2025. Enhancing Generalization in Large-Scale HCVRP: A Rank-Augmented Neural Solver. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25), August 3–7, 2025, Toronto, ON, Canada.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3711896.3736935

## 1 Introduction

The Heterogeneous Capacitated Vehicle Routing Problem (HCVRP) is a well-known class of combinatorial optimization problems. The objective is to determine the most efficient set of routes for a fleet of heterogeneous vehicles with varying capacities and speeds to serve a set of customers with specific demands [7, 17, 18]. HCVRP encompasses two main tasks: partitioning and navigation. Efficient solvers must simultaneously address both the customer partitioning task and the navigation task for customers assigned to each route. While traditional algorithms [3, 8, 19, 23] have significantly contributed to solving a range of such problems, they often struggle with larger-scale instances and require considerable manual design and tuning, which can limit the scalability and adaptability in complex and practical scenarios. Recently, reinforcement learning (RL)-based neural solvers have emerged as a promising alternative to traditional hand-crafted heuristics [14, 15, 21]. Unlike supervised learning [20, 28], which requires a large amount of labeled optimal solutions, RL leverages its ability to explore the environment and learn directly from problem structures, demonstrating strong potential in combinatorial optimization tasks. This approach is particularly advantageous as it relies on cost-effective simulated environments and requires minimal domain expertise for method design. RL-based approaches can be broadly categorized into two main types: constructive methods [12, 14, 15], which sequentially build routing solutions by assigning customers to vehicles, and improvement methods [16, 27], which start with an initial solution and iteratively refine it through optimization operations. In this paper, we focus on constructive methods due to their alignment with real-world scenarios and their computational efficiency in generating feasible solutions from scratch.

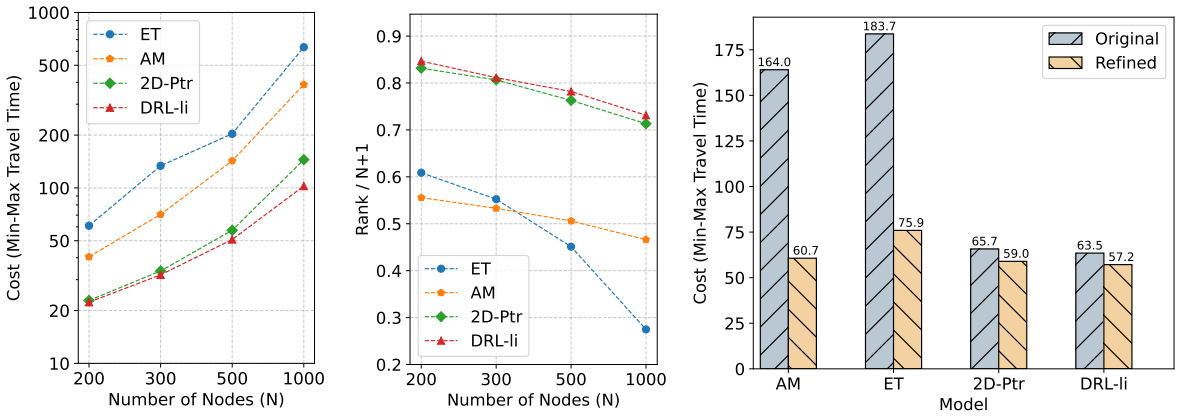



**Figure 1: Empirical study highlighting two findings. The left panel illustrates the performance of different methods (e.g., AM [14], ET [22], 2D-Ptr [18], and DRL-li [17]) across various scales. The middle panel shows the rank of the attention matrix for each method across different scales. The right panel presents a comparison of the performance of different methods before and after the optimization of the navigation task.**

Existing RL-based methods perform well on small-scale HCVRP instances (e.g., $\leq$ 1000 nodes) [2, 17, 18] but face significant challenges when applied to larger-scale instances (e.g., 10,000+ nodes). This is primarily due to the high computational cost associated with training on such large datasets. A potential solution is to train a model on small-scale instances within a reasonable timeframe and then generalize it to large-scale cases. However, when a model trained on small-scale instances is applied to large-scale instances, its performance significantly decreases. Here, we evaluate the performance of four state-of-the-art (SOTA) algorithms [14, 17, 18, 22] across different data scales. Through analysis, we identify two bottlenecks in existing methods that hinder their generalization to large-scale instances:

- **Low Rank of the Attention Matrix:** We calculate the rank of the attention matrix of customer encoder for each SOTA model at different scales, as shown in the middle panel of Fig.1, where $\eta = \frac{RANK}{N+1}$ represents the ratio of the rank of the attention matrix to its full rank (i.e., $N$ denotes the number of customer nodes, and we use $N+1$ where the depot node is also included). We observe a negative correlation between $\eta$ and Min-Max Travel Time among the models. Specifically, models with lower $\eta$ tend to exhibit worse performance. The diminished rank of the attention matrix reduces the diversity of features in large-scale instances, leading to degraded model performance. This low-rank characteristic limits the model's ability to capture complex patterns, particularly in large-scale instances where node differences are more nuanced.
- **Suboptimal Navigation Task:** To identify whether navigation is the primary bottleneck in HCVRP model performance, we employ the four SOTA models to generate initial solutions. We then fix the node set assigned to each vehicle and focus exclusively on optimizing the navigation task by refining the sequence of node visits. As shown in the right panel of Fig.1, optimizing the navigation task leads to significant performance improvements across all algorithms. This suggests that navigation is the key limiting factor in these HCVRP solvers' performance. This limitation likely stems from their reliance on self-attention mechanisms for computing vehicle features, which tend to overemphasize inter-vehicle coordination at the expense of capturing the distinct characteristics of individual vehicles and lead to a deviation from the optimal route.

To address the aforementioned challenges, we introduced two innovations. Firstly, we proposed a simple yet effective approach within the customer encoder to increase the rank upper bound of the attention matrix and approximate its rank upper bound to generate more distinctive and informative node features. This insight stems from our understanding of why existing models struggle to be generalized to large-scale instances, specifically identifying the crucial role of the attention matrix rank in shaping node representations. Secondly, we have proposed a Dual Cross-Attention Module within vehicle encoder, designed to overcome the limitations of existing models in generating suboptimal routes due to excessive feature sharing among vehicles. By leveraging the Dual Cross-Attention Module, our model can better capture the unique attributes of individual vehicles while maintaining overall coordination.

In essence, our contributions in this paper are twofold: 1) we uncovered the underlying reasons for the generalization challenges faced by existing models; and 2) we introduced a novel RL-based neural solver tailored for large-scale data, which demonstrates superior performance and efficiency compared to existing methods.

## 2 Related Work

In recent years, advancements in Vehicle Routing Problems (VRPs) have been driven by neural combinatorial optimization, particularly attention-based neural solvers [12, 14, 15, 29]. While existing neural solvers perform well on small-scale problems, they face

substantial challenges in generalizing to large-scale instances or diverse data distributions. To address this limitation, recent studies have proposed various strategies to enhance scalability and generalization [4–6, 25, 30, 31]. One notable approach is the use of ensemble methods, such as ELG [6]. This method strategically combines heuristic expertise with neural solvers, achieving a balance between exploration and exploitation. This hybrid technique improves generalization on various scales of problems. Another promising development is INViT [5], which significantly reduces the action space through a detailed exploration of high-quality solutions. At the same time, it aggregates a broader range of node features by employing nested views to minimize interference from irrelevant nodes, making each decision step more robust. INViT has demonstrated stable and excellent performance across various distributions of CVRP instances. In TSP/CVRP problems, distance is a crucial factor influencing node selection. Distance-aware mechanisms [25] reshape the attention scores of nodes in the decoder by leveraging distance information, achieving remarkable results across a wide range of large-scale CVRPLib instances. On the other hand, AAFM [31] directly incorporates distance factors into both the encoder and decoder, utilizing a multi-stage training approach that enables the model to attain improved generalization capabilities. Although these models enhance generalization capabilities in certain VRPs by integrating expert knowledge, they have not significantly improved the accuracy and discriminative power of node representations in high-dimensional spaces through DRL. Instead, these models rely on expert knowledge to compensate for the inadequacies of node representations and to guide the decision-making process. This reliance somewhat limits the potential of DRL in deep learning from being fully realized. Furthermore, most approaches still assume homogeneous fleets and struggle to address the complexities of HCVRP.

To address HCVRP, Li et al. [17] proposed a transformer-based DRL model that incorporates a node encoder and a vehicle decoder. The node encoder represents customer and node information, while the vehicle decoder captures the characteristics of different vehicles, emphasizing fleet heterogeneity. The action space is defined as a collection of vehicle-node pairs, allowing for more flexible decision-making. However, this method requires two distinct steps, vehicle selection and node selection, at each decision point, which increases the risk of getting trapped in local optima. To overcome this limitation, Liu et al. [18] introduced the 2D-Ptr model, which employs a dual encoder design. Unlike Li et al. [17], the 2D-Ptr model prioritizes actions rather than vehicles, enabling simultaneous selection of both a vehicle and a node in a single step. This streamlined design improves scalability and adaptability to varying fleet and customer sizes. PARCO [2] further explores a parallel autoregressive strategy combined with a conflict resolution mechanism to improve reasoning efficiency, eliminate biases in model order selection, and improve solution quality. In addition, PARCO enriches context information by integrating the characteristics of all vehicles and partially constructed routes, enabling more effective global decision making.

However, these works primarily focus on collaboration among vehicles, often neglecting the local route optimization of individual vehicles. This oversight makes them prone to falling into local optima in large-scale scenarios. Moreover, many methods have not explicitly aimed to improve generalization to large-scale instance, leaving room for further innovation to achieve robust generalization across HCVRP and real-world scenarios.

# 3 Preliminary

## 3.1 Problem Description

The HCVRP consists of $M$ heterogeneous vehicles, each with a distinct capacity $\rho_i$ and speed $\chi_i$. These vehicles depart from a depot (indexed as 0, located at coordinates $(b_0, c_0)$ with demand $\delta_0 = 0$) to serve $N$ customers (indexed from 1 to $N$), each with specific coordinates $(b_j, c_j)$ and demand $\delta_j$. In HCVRP, each customer must be visited exactly once, and the amount of demand satisfied by a single vehicle in a trip cannot exceed its capacity. Vehicles may return to the depot for reloading.

Let $\mathcal{T} = \{\tau^i\}_{i=1}^{M}$ represent a solution to an instance, where $\tau^i$ denotes the constructed route for vehicle $i$, which starts and ends at a depot. The optimal solution $\mathcal{T}^*$ minimizes the maximum travel time of all vehicles, also known as the min-max objectives [1, 13, 24]. Specifically, let $dist(i, j)$ denote the distance from node $i$ to node $j$, we have

$$\mathcal{T}^* = \arg\min_{\mathcal{T}} \max_{\tau^i \in \mathcal{T}} \left( \sum_{k=2}^{|\tau^i|} \frac{dist(\tau^i_{k-1}, \tau^i_k)}{\chi_i} \right), \tag{1}$$

where $|\tau^i|$ denotes the length of $\tau^i$, and $\tau^i_k$ represents the $k$-th term of $\tau^i$. We are interested in obtaining the optimal solution $\mathcal{T}^*$ by discovering a stochastic policy $\pi_\theta$, which sequentially generates the $t$-th action $(i, j)$, where $i$ represents the index of the vehicle and $j$ represents the index of the selected node (either an unvisited customer or the depot). Here, $\theta$ denotes the trainable parameters.

In this paper, we focus on constructive methods due to their alignment with real-world scenarios and their computational efficiency in generating feasible solutions from scratch. The overall architecture of our method, which includes a Node Encoder, a Vehicle Encoder and an Action Decoder, is illustrated in Fig.2.

## 3.2 Modeling HCVRP via DRL

To solve HCVRP using DRL, we reformulate it as a Markov Decision Process (MDP), defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{F}, R)$:

- State ($\mathcal{S}$): The state of vehicle $i$ at step $t$ is $y_i^t = (\rho_i, \hat{\rho}_i^t, \Upsilon_i^t, \chi_i)$ including the full load of the vehicle $\rho_i$, the speed of vehicle $\chi_i$, the accumulated usage of capacity $\hat{\rho}_i^t$ and time $\Upsilon_i^t$. The state $x_j$ of customer node $j$ at each step $t$ can be considered static and is always $(b_j, c_j, \delta_j)$, including the location coordinates $(b_j, c_j)$ and demand $\delta_j$. Note that the depot's state is $x_0 = (b_0, c_0, 0)$ since the depot's demand $\delta_0$ is set to 0. In addition, we also introduce two tag variables $g_j^t \in \{0, 1\}$ and $\ell_i^t \in \{0, 1, \cdots, N\}$. The former represents whether the node $j$ has been visited, and the latter represents the node where the vehicle $i$ is currently located.
- Action ($\mathcal{A}$): An action at step $t$ is to select a tuple $a^t = (i, j)$ from all the action spaces $\mathcal{A} \in \mathbb{R}^{M \times (N+1)}$, representing that the vehicle $i$ has selected the customer or depot node $j$ at step $t$. In our work, the action space of large-scale instances is greatly reduced, which helps model training and reasoning converge quickly.

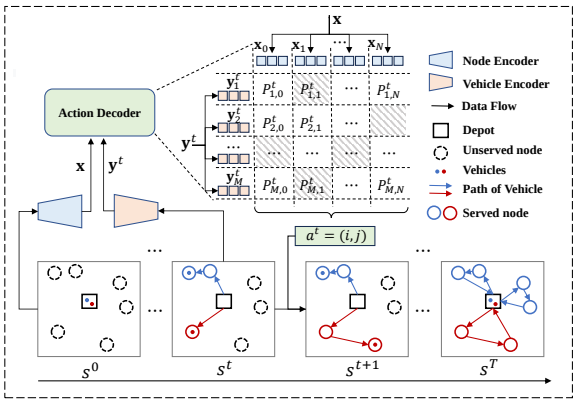



**Figure 2: Architecture of our constructive method.**

- Transition ($\mathcal{F}$): The state transition function deterministically updates environment features based on the selected action. Since the features of customer nodes remain unchanged, the transition occurs mainly in features of the vehicle. After executing the action $a^t = (i, j)$, the status feature of vehicle $i$ changes from $x_i^t$ to $x_i^{t+1}$. The states of vehicles that do not make an action remain unchanged. The specific state transition function is expressed as follows:

$$\hat{\rho}_i^{t+1} = \hat{\rho}_i^t + \delta_j, \tag{2}$$

$$\Upsilon_i^{t+1} = \Upsilon_i^t + \frac{dist(\ell_i^t, j)}{\chi_i}, \tag{3}$$

where $dist(\ell_i^t, j)$ represents the distance between the position ($\ell_i^t$) of the vehicle $i$ and the node $j$. In addition, the two tag variables should also be updated accordingly, i.e., $g_j^t = 1$ and $\ell_i^t = j$.

- Reward ($R$): The reward function is designed to guide the agent toward feasible and cost-effective solutions. Reward of our work is defined as the negative value of the maximum travel time of all vehicles:

$$R = \underset{i \in \{1,2...M\}}{-max} \Upsilon_i, \tag{4}$$

where $\Upsilon_i$ is the total travel time of vehicle $i$. This formulation aligns with the min-max objective of the HCVRP, encouraging the model to minimize the maximum travel time across all vehicles.

## 4 Methodology

We formulate the solution construction for our Rank-Augmented Neural Solver (RANS) using an autoregressive alternating approach [18]. Specifically, given the initial state $s^0$ of a certain HCVRP instance, we define the solution construction process as an action sequence $\{a^t\}_{t=1}^T$, which is described as follows:

$$\mathbf{x} = f^n(s^0; \theta_1), \tag{5}$$

$$\{\mathbf{y}^t, s^t\} = f^v(s^{t-1}, a^{t-1}, \mathbf{x}; \theta_2), \tag{6}$$

$$a^t = f^d(\mathbf{y}^t, \mathbf{x}). \tag{7}$$

Here, the policy $\pi_\theta$ is divided into three parts, i.e., $\pi_\theta = \{f^n, f^v, f^d\}$ and $\theta = \{\theta_1, \theta_2\}$, where $f^n$ denotes the node encoder parameterized by $\theta_1$, which maps the raw features of nodes to their latent embeddings $\mathbf{x}$; $f^v$ represents the vehicle encoder parameterized by $\theta_2$, which iteratively computes the vehicle embedding $\mathbf{y}^t$ at each step $t$; and $f^d$ is the action decoder without parameters, as illustrated in Fig. 2.

Let $\beta_{i,j}^t$ represent the attention score between vehicle $i$ and node $j$ at step $t$, we have

$$\beta_{i,j}^t = \begin{cases} -\infty, & \text{if } g_j^t = 1 \wedge j \neq 0 \\ -\infty, & \text{if } \ell_i^t = 0 \wedge j = 0 \\ -\infty, & \text{if } \rho_i - \hat{\rho}_i^t < \delta_j \\ -\infty, & \text{if } j \notin KNN(\ell_i^t) \\ \lambda tanh(\frac{\mathbf{y}_i^t \mathbf{x}_j}{\sqrt{d}}), & \text{otherwise,} \end{cases} \tag{8}$$

where $d$ is the dimension of node or vehicle embeddings; $tanh(\cdot)$ is the activation function; and $\lambda$ (set to 10 in this work) is a manually set hyper-parameter.

**Invalid Action Masking:** In the action space of size $M \times (N+1)$, certain actions are invalid at step $t$ and must be masked. These include:

- **Visited Customers:** Selecting a customer $j$ that has already been visited before step $t$ (i.e., $g_j^t = 0 \wedge j \neq 0$);
- **Depot Revisits:** Selecting the depot when the vehicle is already at the depot (i.e., $\ell_i^t = 0 \wedge j = 0$);
- **Capacity Constraints:** Selecting a node $j$ whose demand $\delta_j$ exceeds the remaining capacity of vehicle $i$ (i.e., $\rho_i - \hat{\rho}_i^t < \delta_j$);
- **Neighborhood Constraints:** Selecting a node $j$ that is not within the $k$-nearest neighbors (set to 80 in this work) of the last visited node by vehicle $i$ ( i.e., $j \notin KNN(\ell_i^t)$).

The last constraint (KNN-based masking) helps reduce the search space, significantly improving the inference speed of the model [5].

The probability of selecting a tuple $(i, j)$ at step $t$ is computed using the policy network $\pi_\theta$ as follows:

$$P_{i,j}^t = \frac{exp(\beta_{i,j}^t)}{\sum_{i,j} exp(\beta_{i,j}^t)}. \tag{9}$$

The action $a^t = (i, j)$ can be chosen either greedily (by selecting the maximum probability) or by sampling according to the probability distribution $P^t$.

The above steps are repeated until all customers have been visited, resulting in an action sequence $\{a^t\}_{t=1}^T$. This sequence is used to construct the final solution $\mathcal{T}$. Below, we provide a detailed description of the Node Encoder and Vehicle Encoder.

### 4.1 Node Encoder

We begin by mapping the raw features $\mathbf{x}_j^0 = (b_j, c_j, \delta_j)$ of each node $j$ to a new space through a linear layer. Specifically, the transformed features $\mathbf{x}_j^1 \in \mathbb{R}^d$ are computed as follows:

$$\mathbf{x}_j^1 = \begin{cases} \mathbf{x}_j^0 W + DT, & \text{if } j = 0 \\ \mathbf{x}_j^0 W, & \text{otherwise,} \end{cases} \tag{10}$$

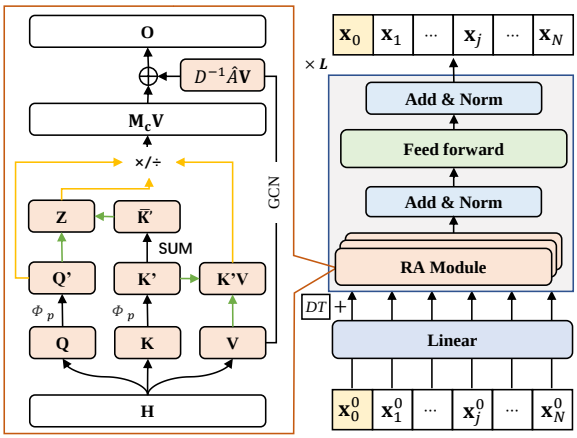



**Figure 3: Node Encoder with Rank-Augmented Modules**

where $W \in \mathbb{R}^{3\times d}$ and $DT \in \mathbb{R}^d$ are trainable parameters. Here, $DT$ is referred to as the depot token, which is introduced to distinguish the depot from customer during training.

Let $\mathbf{H}^1 = \{\mathbf{x}_0^1, \cdots, \mathbf{x}_N^1\}$ are transformed into $\mathbf{H}^L$ through $L$ identical layers, each consisting of two sublayers: a **Multi-Head Rank-Augmented Module (MHR)** sublayer and a **Feed Forward (FF)** sublayer. Both sublayers incorporate residual connections [10] and batch normalization [11] added. The layer-wise propagation rules are defined as follows:

$$\hat{\mathbf{H}}^l = BN\left(\mathbf{H}^l + MHR\left(\mathbf{H}^l, \mathbf{H}^l, \mathbf{H}^l\right)\right), \tag{11}$$

$$\mathbf{H}^{l+1} = BN\left(\hat{\mathbf{H}}^l + FF\left(\hat{\mathbf{H}}^l\right)\right), \tag{12}$$

where $FF(\cdot)$ is a double-layer network with a ReLU activation function, $BN(\cdot)$ denotes batch normalization, and $MHR(\cdot,\cdot,\cdot)$ represents the multi-head Rank-augmented module. The MHR module is defined as:

$$MHR(Q,K,V) = \left(||_{i=1}^{8} RA_i(Q,K,V)\right) W, \tag{13}$$

where $||$ represents the concatenation operation, $W$ are trainable parameters, and $RA_i$ is the Rank-Augmented Module at the $i$-th head, which will be described in detail later.

The final output of the node encoder, denoted as $\mathbf{x}$, is the set of node embeddings $\mathbf{H}^L$.

**Rank-Augmented Module (RA):** The self-attention mechanism serves as a critical component of the Node Encoder, facilitating effective information propagation and aggregation among nodes. Formally, it is defined as:

$$O_i = \sum_{j=1}^{N} \frac{Sim(Q_i, K_j)}{\sum_{j=1}^{N} Sim(Q_i, K_j)} V_j, \tag{14}$$

where $Q_i$, $K_j$, and $V_j$ represent the query, key and value features, respectively.

To improve the focus ability and reduce the computational complexity, we adopt Focused Linear Attention [9], which adjusts the direction of query and key features to bring similar pairs closer while pushing dissimilar pairs away. Specifically, the similarity function is defined as:

$$Sim(Q_i, K_j) = \phi_p(Q_i)\phi_p(K_j)^T, \tag{15}$$

where $\phi_p(x)$ is a specialized nonlinear focus function, given by:

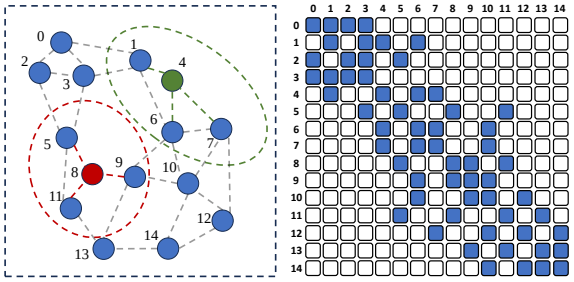
$$\phi_p(x) = \frac{\|ReLU(x)\|}{\|ReLU(x)^{**p}\|} ReLU(x)^{**p}. \tag{16}$$

Here, $\|x\|$ denotes the vector norm of $x$, and $x^{**p}$ represents the element-wise power $p$ of $x$.

The rank of the attention matrix is bounded by the number of tokens (or nodes) $N$ and channel dimension $d$:

$$\begin{aligned} rank\left(\phi(Q)\phi(K)^T\right) &\le min\{rank(\phi(Q)), rank(\phi(K))\} \\ &\le min\{N, d\}. \end{aligned} \tag{17}$$

In common designs, $d$ is typically smaller than $N$, especially in large-scale instances.




(a) KNN Adjacency Graph (b) Normalized Adjacency Matrix

**Figure 4: An example illustrating how the normalized adjacency matrix enhances the rank of the Coefficient Matrix**

Our findings indicate that the diminished rank of the attention matrix limits the diversity of features in large-scale instances, leading to degraded model performance. To address this issue, we propose constructing a coefficient matrix $M_c$ with higher upper bound of rank. A simple yet effective solution is:

$$O = M_cV = \left(\phi(Q)\phi(K)^T + D^{-1}\hat{A}\right)V, \tag{18}$$

where $\hat{A} = A + I_N$ is the adjacency matrix with added self-connections, $D_{ii} = \sum_j \hat{A}_{ij}$, and $I_N$ is the identity matrix. Intuitively, fusing the features of one-hop neighbors can facilitate a clearer distinction of node features. As illustrated in Fig. 4(a), if the latent variables computed for node 4 and node 8 are relatively similar, their distinction can be enhanced by leveraging the features of their nearest $k$ neighbors. Furthermore, $rank(M_c) \le rank\left(\phi(Q)\phi(K)^T\right) + rank(D^{-1}\hat{A})$. Since $D^{-1}\hat{A}$ (see Fig. 4(b)) has the potential to be a full rank matrix, it can practically increase the rank upper bound of $M_c$.

To approximate the rank upper bound of $M_c$ and improve feature diversity, we introduce the Disagreement Regularization (DR), which is expressed as follows:

$$\mathcal{L}_d = \frac{1}{N(N-1)}\left(\sum_{i=1}^{N}\sum_{j=1,j\neq i}^{N} \frac{O_i \cdot O_j}{\|O_i\|\|O_j\|}\right). \tag{19}$$

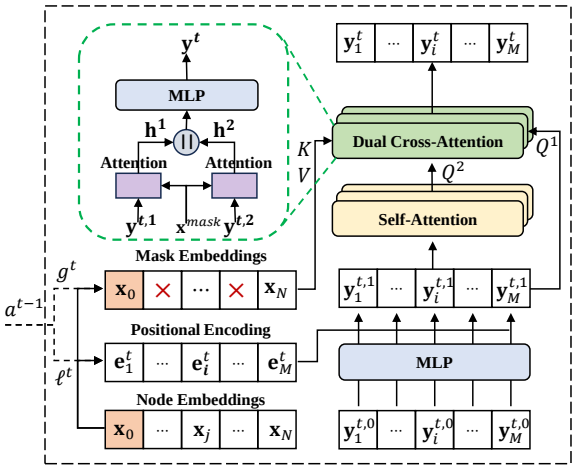



**Figure 5: Vehicle Encoder with Dual Cross-Attention Module**

This loss penalizes redundancy in the $O$, encouraging the model to generate more diverse and discriminative representations.

## 4.2 Vehicle Encoder

Let $\mathbf{y}_i^{t,0} = (\rho_i, \hat{\rho}_i^t, \Upsilon_i^t, \chi_i)$ denote the initial representation of vehicle $i$ at step $t$. Here, $\chi_i$ represents the speed, $\rho_i$ represents the capacity, $\hat{\rho}_i^t$ represents the accumulated usage of capacity up to step $t$, and $\Upsilon_i^t$ denotes the running time. We first update the representation of vehicle embeddings by incorporating its last visited node embeddings as follows:

$$\mathbf{y}_i^{t,1} = MLP(\mathbf{y}_i^{t,0}) + \mathbf{e}_i^t W, \tag{20}$$

where $MLP(\cdot)$ is a trainable Multi-layer Perceptron that maps the initial features to a $d$- dimensional space. Additionally, $\mathbf{e}_i^t$ denotes the positional encoding of the last visited node, obtained by indexing the node embeddings $\mathbf{x}$ with $t_i^t$. Note that $W$ is a learnable weight matrix that transforms the positional encoding to match the dimensionality of the output of the MLP.

Next, we input $\mathbf{y}^{t,1} = \{\mathbf{y}_1^{t,1}, \cdots, \mathbf{y}_M^{t,1}\} \in \mathbb{R}^{M\times d}$ into a self-Attention layer to facilitate information exchange among vehicles as

$$\mathbf{y}^{t,2} = \mathbf{y}^{t,1} + \left(||_{i=1}^{8} ATT_i(\mathbf{y}^{t,1}, \mathbf{y}^{t,1}, \mathbf{y}^{t,1})\right) W, \tag{21}$$

$$ATT_i(Q,K,V) = softmax\left(\frac{[QW_i^Q][KW_i^K]^T}{\sqrt{d'}}\right) VW_i^V, \tag{22}$$

where $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d\times d'}$ are trainable parameters for each head. Moreover, to address the model's poor performance in navigation task, we propose a Dual Cross-Attention module.

**Dual Cross-Attention Module (DCA):** We first construct the mask embeddings $\mathbf{x}^{mask}$ by removing the rows corresponding to served customers $\{j|1 \le j \le N \wedge g_j^t = 1\}$ from $\mathbf{x}$. Next, we compute:

$$\mathbf{h}^1 = ATT(\mathbf{y}^{t,1}, \mathbf{x}^{mask}, \mathbf{x}^{mask}), \tag{23}$$

$$\mathbf{h}^2 = ATT(\mathbf{y}^{t,2}, \mathbf{x}^{mask}, \mathbf{x}^{mask}), \tag{24}$$

where $\mathbf{h}^2$ denotes the vehicle representations after comprehensive coordination, potentially losing some unique characteristics of individual vehicles, while $\mathbf{h}^1$ corresponds to the distinct features of each vehicle. Specifically, $\mathbf{h}^2$ is associated with the partition task, while $\mathbf{h}^1$ is associated with the navigation task. The final vehicle embedding $\mathbf{y}^t$ is obtain by fusing $\mathbf{h}^1$ and $\mathbf{h}^2$, i.e.,

$$\mathbf{y}^t = MLP(\mathbf{h}^1||\mathbf{h}^2). \tag{25}$$

As shown in Fig. 5, the Mask Embeddings and Positional Encoding evolve over time $t$ based on the action $a^{t-1}$, resulting in dynamic updates to the vehicle embeddings $\mathbf{y}^t$.

## 4.3 Training

Considering the impracticality of training many models on large-scale instances, we opted to train on HCVRP instances with 3 vehicles and 40 customers (M3-N40), utilizing a minimal amount of data for fine-tuning. Similarly, the baseline models used for comparison were also trained on M3-N40 and subsequently tested on some large instances. Given that the DRL-Li [17] model could not generalize well across vehicle scales, we tested the DRL-Li models trained on M3-N40, M5-N40, and M7-N40 on the corresponding vehicle scales. Additionally, since PARCO [2] was specifically trained in a mixed manner across various data scales, we directly employed the pre-trained models. RANS[1] was initially trained on HCVRP M3-N40 instances, following the same hyper-parameter configuration as 2D-Ptr [18]. To prevent overfitting on small-scale datasets, we implemented an early stopping mechanism, terminating the training after the 40th epoch to obtain a model with preliminary generalization capabilities. Subsequently, consistent with most NCO models, we fine-tuned the model on slightly larger datasets (N<=100) for 2 epochs. During this phase, the learning rate is 0.00001. Each epoch contained only 217,600 instances. For smaller datasets (M3-N40, M3-N80, M5-N40, M5-N80, M7-N40, M7-N80), the batch size was set to 128, while for larger datasets (M3-N100, M5-N100, M7-N100), the batch size was adjusted to 64. The total data volume of 435,200 in the fine-tuning phase is lower than the 1,280,000 instances used in a single epoch in the initial training phase.

We train RANS via the REINFORCE gradient estimator [26] with a shared baseline as PARCO [2]:

$$\nabla\mathcal{L} = \frac{1}{B\cdot U}\sum_{B}^{i}\sum_{U}^{j} G_{ij}\cdot\nabla\log p_\theta(\mathcal{T}_{ij}|s_i) + \alpha\cdot\nabla\mathcal{L}_d, \tag{26}$$

where $B$ is the size of batch and $U$ is the number of symmetric solutions sampled from problem instance $s_i$; $G_{ij}$ is the advantage $R(\mathcal{T}_{ij}, s_i) - b_{share}(s_i)$ of a solution $\mathcal{T}_{ij}$ compared to the shared baseline $b_{share}(s_i$ of problem instance $s_i$. $\alpha$ is a constant (set to 0.05 in this work) and $\nabla\mathcal{L}_d$ is the Disagreement Regularization.

## 5 Experimental

In this section, we conduct experiments to address the following questions:

- **Q1.** Does the proposed algorithm in this paper exhibit broad generalizability, enabling it to adapt to datasets of various scales, particularly large-scale datasets?
- **Q2.** Can the Rank-augmented Module enhance the upper bound of the rank of the attention matrix in the node encoder? Does the Disagreement Regularization assist the model

[1] https://github.com/AAI-ZZU/RANS

in approximating the rank of the attention matrix to its upper bound?
- **Q3.** Does the Dual Cross-Attention Module in the Vehicle Encoder optimize the navigation task in the HCVRP problem, thereby comprehensively improving the model's performance?

## 5.1 Comparative Study

To address the first research question (**Q1**), we conducted a series of comparative experiments involving one heuristic method, **SISR (2020)** [3], and five SOTA DRL-based neural solvers, including **AM (2018)** [14], **DRL-Li (2022)** [17], **ET (2023)** [22], **2D-Ptr (2024)** [18], and **PARCO (2024)** [2]. It is worth noting that AM and ET were originally designed for the Capacitated Vehicle Routing Problem (CVRP). To adapt these methods to HCVRP, we followed the modifications proposed in [18], ensuring their applicability to the target problem. All experiments are performed on servers equipped with RTX3090 GPUs.

*5.1.1 Performance Evaluation on HCVRP Synthetic Dataset.* To comprehensively evaluate model performance, we constructed diverse datasets with varying problem scales and fleet configurations. The experimental setup encompasses 18 distinct problem settings, systematically defined by the number of vehicles ($M \in \{3, 5, 7\}$) and customers ($N \in \{200, 500, 1000, 3000, 5000, 10000\}$). Each dataset configuration contains 128 and 24 HCVRP instances for $N = \{200, 500, 1000\}$ and $N = \{3000, 5000, 10000\}$, respectively. In all cases, customer and depot coordinates are uniformly sampled from a unit grid $[0, 1] \times [0, 1]$. Customer demands are drawn uniformly from the discrete set $\{1, 2, \cdots, 9\}$, while the depot's demand is fixed at 0. To ensure realistic fleet heterogeneity, vehicle capacities are randomly sampled from the discrete set $\{20, 21, ..., 40\}$, and their speeds are uniformly sampled from the range $[0.5, 1]$.

As shown in Tab. 1 and Tab. 2, the experimental results reveal several key findings: 1) The heuristic-based SISR method demonstrates superior solution quality on small-scale instances, albeit at the cost of significantly higher computational time. In contrast, the DRL-based methods exhibit substantially faster inference but with compromised solution quality. 2) The symmetry-based augmentation approach outperforms the greedy sampling method in terms of performance, requiring only marginally more computation time while maintaining a significant advantage over SISR in computational efficiency. 3) Our proposed model RANS achieves near-optimal performance, closely trailing SISR with minimal optimality gaps while distinctly outperforming other DRL-based neural solvers. The superiority of RANS is particularly pronounced on large-scale instances, where it achieves substantial performance improvements over DRL-based baselines. Notably, in the most challenging scenario with 7 vehicles and 10,000 nodes, RANS outperforms the best-performing DRL-based competitor by a remarkable margin of approximately 48.5 in terms of minimum-maximum travel time.

*5.1.2 Performance Evaluation on CVRP Benchmark Dataset.* We further conducted experiments on several datasets from the CVRPLib[2] (Capacitated Vehicle Routing Problem Library) to evaluate the generalization capability of our model across diverse real-world application scenarios. CVRPLib is a widely recognized repository of benchmark instances for the Capacitated Vehicle Routing Problem (CVRP) and its variants, providing a robust platform for testing algorithmic performance under realistic conditions. The experimental results, as presented in Tab. 3, demonstrate that RANS consistently outperforms other DRL-based methods across all dataset sizes, maintaining a narrow gap relative to the Best Known Solutions (BKS) provided by the library.

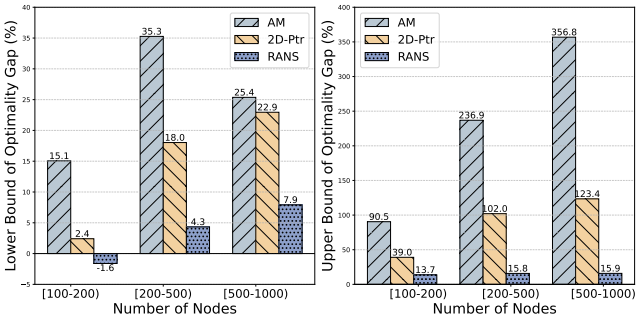



**Figure 6: The minimum and maximum gaps for some baselines across various CVRPLib datasets.**

We also notice that the performance of all models on CVRPLib is generally inferior to their performance on randomly generated test data. This discrepancy can be attributed to the fact that the data distribution in CVRPLib does not follow a normal distribution but instead reflects the irregular and heterogeneous customer distributions commonly observed in real world scenarios. These results underscore the strong generalization capability of our model, particularly in handling complex and realistic problem instances.

Furthermore, Fig. 6 illustrates the minimum and maximum gaps between the baseline and RANS with respect to the BKS for the selected CVRPLib datasets. The results clearly indicate that RANS achieves superior performance across both metrics, outperforming all baseline methods. Notably, on certain CVRPLib datasets with scales ranging from 100 to 200 nodes, RANS even surpasses the performance of BKS in some cases. This exceptional performance further highlights the remarkable generalizability of RANS, solidifying its potential for solving practical routing problems. For comprehensive performance evaluations of each model on CVRPLib, please refer to Appendix B.

## 5.2 Ablation Study

In this subsection, we perform ablation studies on both the node encoder and the vehicle encoder to investigate and address the second (**Q2**) and third (**Q3**) research questions, respectively.

*5.2.1 Ablation Study on Node Encoder.* We first computed the rank of the matrix $\phi(Q)\phi(K)^T$, referred to as linear attention. Subsequently, we calculated the rank of the matrix $\phi(Q)\phi(K)^T + D^{-1}\hat{A}$, where $D^{-1}\hat{A}$ represents the normalized adjacency matrix derived from the $K$-nearest neighbors. We conducted these computations both with and without Disagreement Regularizations, as illustrated in Fig. 7. The experimental results indicate that in the absence of the normalized adjacency matrix (i.e., using only linear attention),

[2] http://vrp.atd-lab.inf.puc-rio.br/index.php/en/

**Table 1: Performance on small-scale HCVRPs. Here, (*g.*) refers to greedy performance while (*aug*×*8*) refers to the augmentation with 8 instances following [15]. The average inference times are shown in parentheses (·).**

| *N* | 200 | | | 500 | | | 1000 | | | Gap (%) |
|---|---|---|---|---|---|---|---|---|---|---|
| *M* | 3 | 5 | 7 | 3 | 5 | 7 | 3 | 5 | 7 | avg |
| SISR | 19.33 (0.73h) | 11.63 (0.69h) | 8.36 (0.67h) | 44.08 (3.04h) | 26.96 (3.52h) | 19.79 (3.27h) | 86.71 (13.66h) | 52.96 (14.71h) | 37.33 (15.31h) | 0.00 |
| AM (*g.*) | 40.50 (0.40s) | 19.96 (0.56s) | 11.93 (0.36s) | 142.70 (1.07s) | 74.54 (0.98s) | 47.35 (0.96s) | 388.30 (2.38s) | 199.79 (2.11s) | 129.41 (2.06s) | 168.86 |
| ET (*g.*) | 145.57 (0.76s) | 111.62 (0.69s) | 79.66 (0.66s) | 483.16 (2.07s) | 400.65 (1.87s) | 357.01 (1.79s) | 1006.62 (4.15s) | 831.37 (3.83s) | 749.59 (3.66s) | >500 |
| DRL-Li (*g.*) | 22.31 (0.88s) | 14.11 (0.83s) | 10.63 (0.89s) | 50.80 (4.09s) | 31.51 (4.01s) | 24.05 (4.09s) | 102.45 (14.49s) | 61.83 (14.70s) | 47.40 (14.74s) | 19.93 |
| 2D-Ptr (*g.*) | 22.83 (0.58s) | 13.36 (0.57s) | 9.34 (0.57s) | 57.35 (1.31s) | 31.86 (1.20s) | 22.25 (1.27s) | 144.88 (2.62s) | 72.55 (2.44s) | 46.29 (2.38s) | 25.95 |
| PARCO (*g.*) | 23.04 (0.54s) | 12.41 (0.53s) | **8.77** (0.50s) | 117.02 (1.43s) | 61.04 (1.26s) | 23.74 (1.20s) | 483.76 (2.77s) | 331.37 (2.37s) | 203.64 (2.18s) | 411.78 |
| RANS (*g.*) | **20.50** (0.68s) | **12.41** (0.65s) | 8.82 (0.66s) | **46.65** (1.64s) | **27.78** (1.66s) | **19.98** (1.61s) | **92.02** (3.24s) | **53.65** (3.26s) | **38.41** (3.25s) | **4.26** |
| AM (*aug*×*8*) | 31.88 (0.45s) | 16.63 (0.59s) | 11.19 (0.40s) | 108.46 (1.20s) | 56.93 (1.18s) | 35.71 (1.21s) | 304.25 (3.46s) | 152.12 (3.19s) | 95.49 (3.01s) | 119.26 |
| ET (*aug*×*8*) | 117.59 (0.82s) | 328.30 (0.75s) | 54.49 (0.72s) | 446.26 (2.22s) | 83.53 (1.97s) | 54.49 (1.94s) | 938.96 (4.50s) | 690.37 (4.21s) | 547.00 (4.06s) | >500 |
| DRL-Li(*aug*×*8*) | 21.82 (0.93s) | 13.77 (0.87s) | 10.25 (0.92s) | 49.92 (5.13s) | 30.98 (5.27s) | 23.33 (5.06s) | 100.28 (37.44s) | 60.73 (37.53s) | 45.54 (38.06s) | 16.92 |
| 2D-Ptr (*aug*×*8*) | 22.23 (0.61s) | 13.08 (0.62s) | 9.14 (0.60s) | 53.38 (1.46s) | 30.90 (1.49s) | 21.80 (1.43s) | 110.80 (3.68s) | 62.83 (3.76s) | 43.97 (3.69s) | 19.85 |
| PARCO (*aug*×*8*) | 21.44 (0.58s) | **12.21 (0.57s)** | **8.64 (0.56s)** | 142.53 (1.50s) | 49.01 (1.34s) | 22.22 (1.28s) | 407.86 (3.27s) | 253.03 (3.17s) | 151.27 (2.98s) | 347.78 |
| RANS (*aug*×*8*) | **20.27** (0.75s) | 12.25 (0.74s) | 8.70 (0.73s) | **46.18** (1.71s) | **27.56** (1.71s) | **19.80** (1.73s) | **91.19** (4.32s) | **53.22** (4.10s) | **38.14** (4.01s) | **3.31** |

**Table 2: Performance on large-scale HCVRPs. † For 10,000-node instances, the number of augmentation instances is reduced to 4 to avoid memory overflow. ‡ The computational complexity of SISR on large-scale instances is prohibitive (≥100h).**

| *N* | 3000 | | | 5000 | | | 10000† | | |
|---|---|---|---|---|---|---|---|---|---|
| *M* | 3 | 5 | 7 | 3 | 5 | 7 | 3 | 5 | 7 |
| SISR‡ | - | - | - | - | - | - | - | - | - |
| AM (*g.*) | 1330.11 (6.54s) | 818.60 (6.56s) | 587.60 (6.08s) | 2408.78 (11.41s) | 1511.95 (10.97s) | 1051.35 (10.69s) | 5125.30 (23.18s) | 3025.45 (22.47s) | 2241.65 (22.06s) |
| ET (*g.*) | 3067.06 (13.31s) | 2712.62 (12.52s) | 2897.33 (12.43s) | 5023.63 (24.62s) | 4636.06 (23.78s) | 4814.30 (23.20s) | 10134.00 (73.36s) | 9670.77 (73.55s) | 9801.49 (72.43s) |
| DRL-Li (*g.*) | 330.63 (23.58s) | 187.34 (34.68s) | 143.65 (46.98s) | 593.46 (60.54s) | 317.27 (91.17s) | 239.77 (126.92s) | 1261.40 (255.25s) | 680.16 (379.23s) | 511.24 (531.84s) |
| 2D-Ptr (*g.*) | 632.29 (6.20s) | 351.29 (6.01s) | 243.35 (5.98s) | 1294.64 (10.33s) | 767.66 (10.27s) | 526.61 (10.27s) | 3005.32 (21.24s) | 1832.14 (21.32s) | 1429.48 (20.93s) |
| PARCO (*g.*) | 1674.69 (7.61s) | 1192.65 (5.33s) | 1087.71 (5.50s) | 2785.76 (13.07s) | 1972.26 (9.93s) | 1735.41 (8.57s) | 6003.29 (28.71s) | 4589.66 (21.40s) | 3721.28 (17.79s) |
| RANS (*g.*) | **276.77** (10.50s) | **155.92** (10.18s) | **112.51** (10.29s) | **456.69** (17.07s) | **270.20** (17.18s) | **189.46** (17.24s) | **962.19** (34.25s) | **634.70** (34.68s) | **462.74** (34.62s) |
| AM (*aug*×*8*) | 1154.56 (7.01s) | 713.52 (6.84s) | 505.67 (6.61s) | 2025.00 (12.37s) | 1225.32 (12.07s) | 905.75 (11.73s) | 4601.44 (27.73s) | 2725.78 (31.27s) | 1992.67 (27.09s) |
| ET (*aug*×*8*) | 2794.21 (24.12s) | 2358.60 (23.53s) | 2278.85 (23.69s) | 4586.02 (64.81s) | 3805.02 (64.44s) | 3805.01 (64.44s) | 9650.18 (157.98s) | 8361.11 (148.88s) | 8712.39 (152.22s) |
| DRL-Li (*aug*×*8*) | 311.32 (25.20s) | 180.68 (36.06s) | 136.73 (49.19s) | 535.41 (65.58s) | 307.64 (98.47s) | 226.93 (136.91s) | 1107.24 (326.90s) | 665.00 (509.09s) | 496.73 (726.02s) |
| 2D-Ptr (*aug*×*8*) | 474.31 (9.59s) | 228.18 (9.54s) | 151.35 (9.54s) | 1057.59 (15.03s) | 522.08 (14.89s) | 306.84 (14.83s) | 2513.06 (23.82s) | 1466.85 (23.75s) | 950.10 (24.72s) |
| PARCO (*aug*×*8*) | 1416.61 (9.00s) | 902.75 (9.13s) | 759.53 (8.94s) | 2446.98 (15.32s) | 1507.41 (11.99s) | 1207.50 (13.20s) | 5304.41 (30.89s) | 3414.67 (24.09s) | 2720.03 (20.31s) |
| RANS (*aug*×*8*) | **266.00** (16.75s) | **153.61** (16.74s) | **111.06** (16.73s) | **431.50** (27.04s) | **254.29** (27.28s) | **182.74** (27.21s) | **889.05** (47.32s) | **551.54** (47.10s) | **387.50** (47.81s) |

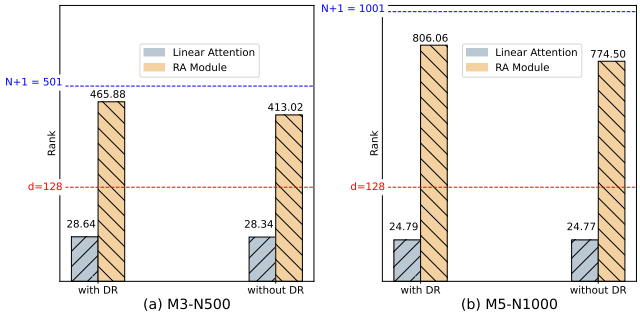



**Figure 7: Variation in the Rank of Attention Matrices Across Different Configurations on M3-N500 and M5-N1000.**

the rank of the attention matrix does not exceed $d$, where $d = 128$ denotes the channel dimension. However, with the addition of the normalized adjacency matrix (i.e., the RA module), a significant increase in the rank is observed. Furthermore, the application of Disagreement Regularization further elevates the rank of the attention matrix to its full rank $N$. These findings validate our hypothesis that the RA module effectively increases the upper bound of the attention matrix rank, while Disagreement Regularization aids the model in more closely approaching this upper bound.

*5.2.2 Ablation Study on Vehicle Encoder.* The Dual Cross-Attention Module in the vehicle encoder is designed to enhance the navigation task in HCVRP. To evaluate the impact of this module, we compared two configurations: one without Dual Cross-Attention Module, referred to as "without DCA", which uses standard cross attention,

**Table 3: Performance on CVRPLib. BKS refers to the "Best Known Solution".**

| *N* | 100-200 | | | 200-500 | | | 500-1000 | | | Gap (%) |
|---|---|---|---|---|---|---|---|---|---|---|
| *M* | Len | Gap | Time | Len | Gap | Time | Len | Gap | Time | avg |
| BKS | 22380.04 | 0.00% | - | 52326.36 | 0.00% | - | 103316.26 | 0.00% | - | 0.00% |
| AM (*g.*) | 39684.45 | 77.32% | 0.28s | 126637.05 | 142.01% | 0.70s | 281001.48 | 171.98% | 1.59s | 130.44% |
| ET (*g.*) | 43115.25 | 92.65% | 0.40s | 101168.23 | 93.34% | 0.73s | 204914.73 | 98.33% | 1.61s | 94.77% |
| 2D-Ptr (*g.*) | 28779.66 | 28.60% | 0.31s | 94899.72 | 81.36% | 0.79s | 237830.84 | 130.20% | 1.77s | 80.05% |
| RANS (*g.*) | **24347.93** | **8.08%** | **0.47s** | **57523.33** | **9.93%** | **0.96s** | **114725.21** | **11.05%** | **2.20s** | **9.69%** |
| AM (*aug×8*) | 32553.15 | 45.46% | 0.40s | 94723.88 | 81.05% | 0.82s | 239044.24 | 131.37% | 2.10s | 85.96% |
| ET (*aug×8*) | 38579.00 | 72.38% | 0.46s | 96143.47 | 83.74% | 0.84s | 190124.56 | 98.33% | 2.21s | 84.82% |
| 2D-Ptr (*aug×8*) | 26914.76 | 20.26% | 0.40s | 73770.64 | 40.98% | 0.86s | 174916.77 | 69.30% | 2.23s | 43.51% |
| RANS (*aug×8*) | **23888.65** | **6.74%** | **0.57s** | **56887.39** | **8.72%** | **1.14s** | **114026.99** | **10.37%** | **2.68s** | **8.61%** |

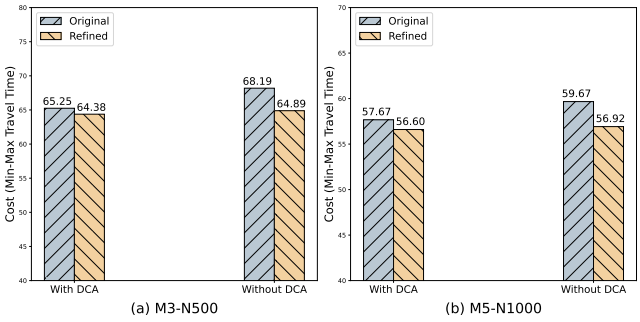



**Figure 8: A comparison of the performance of different configurations before and after the optimization of the navigation task on M3-N500 and M5-N1000 datasets.**

and another with Dual Cross-Attention Module, referred to as "with DCA". Additionally, to assess the potential improvement of RANS in the navigation task, we first obtained an initial solution using RANS and then optimized the visiting sequence for each vehicle using SISR while keeping the node assignments unchanged. The experimental results are illustrated in Fig. 8.

From the Fig. 8, we observe the following: 1) The model's performance declines when the DCA module is removed; 2) Both "with DCA" and "without DCA" exhibit a gap compared to the refined solution achieved by SISR optimization, but the gap is significantly smaller for "with DCA"; 3) There is a slight but noticeable difference between the refined solutions of "with DCA" and "without DCA". These results demonstrate that the Dual Cross-Attention Module plays a critical role in the vehicle encoder, not only aiding in the optimization of the navigation task but also contributing to the improvement of the partition task to some extent.

## 6 Conclusion

In this paper, we present two key findings: 1) the rank of the attention matrix exhibits a positive correlation with model performance, and 2) existing models often underperform in navigation tasks. Through analysis, we identify two critical bottlenecks that hinder the scalability of existing algorithms to large-scale instances: First, the low rank of the attention matrix leads to over-smoothing of node features, which compromises the accuracy of the Markov decision process; Second, the reliance on self-attention mechanisms for inter-vehicle information exchange results in vehicle representations that lack distinctiveness, thereby degrading navigation task performance. To address these challenges, we propose two novel solutions. First, we introduce a Rank-augmented Module, a simple yet effective approach to increase the upper bound of the attention matrix rank, complemented by a Disagreement Regularization to drive the rank closer to this upper bound. Second, we design the Dual Cross-Attention Module, which enables the vehicle encoder to generate vehicle representations that preserve their unique characteristics while incorporating information from other vehicles, significantly enhancing navigation task performance in HCVRP. Extensive experiments on various synthetic and benchmark datasets demonstrate that our method effectively increases the attention matrix rank and exhibits strong scalability. Notably, our approach outperforms existing algorithms in efficiency on large-scale instances. Future work will explore extending these findings to other variants of vehicle routing problems or combinatorial optimization problems to further validate their generalizability.

## Acknowledgments

This research is supported by National Natural Science Foundation of China (Grant No. 62276238, U24A20326, 62325602, 62036010), the Foundation of Henan Educational Committee under Grant (No.25HASTIT034), the Natural Science Foundation of Henan under Grant (No.232300421095). This research is also supported by the National Research Foundation, Singapore under its AI Singapore Programme (AISG Award No: AISG3-RP-2022-031).

## References


[1] Luca Bertazzi, Bruce Golden, and Xingyin Wang. 2015. Min–Max vs. Min–Sum Vehicle Routing: A worst-case analysis. *European Journal of Operational Research* 240, 2 (2015), 372–381. doi:10.1016/j.ejor.2014.07.025

[2] Federico Berto, Chuanbo Hua, Laurin Luttmann, Jiwoo Son, Junyoung Park, Kyuree Ahn, Changhyun Kwon, Lin Xie, and Jinkyoo Park. 2024. Parco: Learning parallel autoregressive policies for efficient multi-agent combinatorial optimization. *arXiv preprint arXiv:2409.03811* (2024).

[3] Jan Christiaens and Greet Vanden Berghe. 2020. Slack Induction by String Removals for Vehicle Routing Problems. *Transportation Science* (Mar 2020), 417–433. doi:10.1287/trsc.2019.0914

[4] Darko Drakulic, Sofia Michel, Florian Mai, Arnaud Sors, and Jean-Marc Andreoli. 2023. BQ-NCO: Bisimulation Quotienting for Generalizable Neural Combinatorial Optimization. *arXiv preprint arXiv:2409.03811* (Jan 2023).

[5] Han Fang, Zhihao Song, Paul Weng, and Yutong Ban. 2024. INViT: A Generalizable Routing Problem Solver with Invariant Nested View Transformer. arXiv:2402.02317 [cs.LG] https://arxiv.org/abs/2402.02317

[6] Chengrui Gao, Haopu Shang, Ke Xue, Dong Li, and Chao Qian. 2023. Towards generalizable neural solvers for vehicle routing problems via ensemble with transferrable local policy. *arXiv preprint arXiv:2308.14104* (2023).

[7] Bruce Golden, Arjang Assad, Larry Levy, and Filip Gheysens. 1984. The fleet size and mix vehicle routing problem. *Computers & Operations Research* 11, 1 (1984), 49–66.

[8] M. Haimovich and A. H. G. Rinnooy Kan. 1985. Bounds and Heuristics for Capacitated Routing Problems. *Mathematics of Operations Research* (Nov 1985), 527–542. doi:10.1287/moor.10.4.527

[9] Dongchen Han, Xuran Pan, Yizeng Han, Shiji Song, and Gao Huang. 2023. Flatten transformer: Vision transformer using focused linear attention. In *Proceedings of the IEEE/CVF international conference on computer vision*. 5961–5971.

[10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. IEEE Computer Society, 770–778. doi:10.1109/CVPR.2016.90

[11] Sergey Ioffe and Christian Szegedy. 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*. pmlr, 448–456.

[12] Yan Jin, Yuandong Ding, Xuanhao Pan, Kun He, Li Zhao, Tao Qin, Lei Song, and Jiang Bian. 2023. Pointerformer: Deep reinforced multi-pointer transformer for the traveling salesman problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 8132–8140.

[13] Minjun Kim, Junyoung Park, and Jinkyoo Park. 2023. Learning to CROSS exchange to solve min-max vehicle routing problems. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/forum?id=ZcnzsHC10Y

[14] Wouter Kool, Herkevan Hoof, and Max Welling. 2018. Attention, Learn to Solve Routing Problems! *arXiv: Machine Learning,arXiv: Machine Learning* (Mar 2018).

[15] Yeong-Dae Kwon, Jinho Choo, Byoungjip Kim, Iljoo Yoon, Youngjune Gwon, and Seung-Jai Min. 2020. POMO: Policy Optimization with Multiple Optima for Reinforcement Learning. *Neural Information Processing Systems,Neural Information Processing Systems* (Jan 2020).

[16] Jingwen Li, Yining Ma, Zhiguang Cao, Yaoxin Wu, Wen Song, Jie Zhang, and Yeow Meng Chee. 2024. Learning Feature Embedding Refiner for Solving Vehicle Routing Problems. *IEEE Transactions on Neural Networks and Learning Systems* 35, 11 (2024), 15279–15291. doi:10.1109/TNNLS.2023.3285077

[17] Jingwen Li, Yining Ma, Ruize Gao, Zhiguang Cao, Andrew Lim, Wen Song, and Jie Zhang. 2022. Deep Reinforcement Learning for Solving the Heterogeneous Capacitated Vehicle Routing Problem. *IEEE Transactions on Cybernetics* (Dec 2022), 13572–13585. doi:10.1109/tcyb.2021.3111082

[18] Qidong Liu, Chaoyue Liu, Shaoyao Niu, Cheng Long, Jie Zhang, and Mingliang Xu. 2024. 2D-Ptr: 2D Array Pointer Network for Solving the Heterogeneous Capacitated Vehicle Routing Problem. In *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*. 1238–1246.

[19] Jens Lysgaard, Adam N. Letchford, and Richard W. Eglese. 2004. A new branch-and-cut algorithm for the capacitated vehicle routing problem. *Mathematical Programming* (Jun 2004), 423–445. doi:10.1007/s10107-003-0481-8

[20] He Lyu, Ningyu Sha, Shuyang Qin, Ming Yan, Yuying Xie, and Rongrong Wang. 2019. Advances in neural information processing systems. *Advances in neural information processing systems* 32 (2019).

[21] Mohammadreza Nazari, Afshin Oroojlooy, Lawrence Snyder, and Martin Takáč. 2018. Reinforcement Learning for Solving the Vehicle Routing Problem. *arXiv: Artificial Intelligence,arXiv: Artificial Intelligence* (Feb 2018).

[22] Jiwoo Son, Minsu Kim, Sanghyeok Choi, Hyeonah Kim, and Jinkyoo Park. 2023. Solving np-hard min-max routing problems as sequential generation with equity context. In *ICML 2023 Workshop: Sampling and Optimization in Discrete Space*.

[23] Thibaut Vidal. 2022. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Computers & Operations Research* (Apr 2022), 105643. doi:10.1016/j.cor.2021.105643

[24] Xingyin Wang, Bruce Golden, Edward Wasil, and Rui Zhang. 2016. The min–max split delivery multi-depot vehicle routing problem with minimum service time requirement. *Computers & Operations Research* 71 (2016), 110–126. doi:10.1016/j.cor.2016.01.008

[25] Yang Wang, Ya-Hui Jia, Wei-Neng Chen, and Yi Mei. 2024. Distance-aware Attention Reshaping: Enhance Generalization of Neural Solver for Large-scale Vehicle Routing Problems. arXiv:2401.06979 [cs.AI]

[26] Ronald J. Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning* (May 1992), 229–256. doi:10.1007/bf00992696

[27] Yaoxin Wu, Wen Song, Zhiguang Cao, Jie Zhang, and Andrew Lim. 2019. Learning Improvement Heuristics for Solving Routing Problems. *IEEE Transactions on Neural Networks,IEEE Transactions on Neural Networks* (Dec 2019).

[28] Yubin Xiao, Di Wang, Boyang Li, Mingzhao Wang, Xuan Wu, Changliang Zhou, and You Zhou. 2024. Distilling autoregressive models to obtain high-performance non-autoregressive solvers for vehicle routing problems with faster inference speed. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 20274–20283.

[29] Liang Xin, Wen Song, Zhiguang Cao, and Jie Zhang. 2021. Multi-decoder attention model with embedding glimpse for solving vehicle routing problems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 12042–12049.

[30] Haoran Ye, Jiarui Wang, Helan Liang, Zhiguang Cao, Yong Li, and Fanzhang Li. 2024. GLOP: Learning Global Partition and Local Construction for Solving Large-scale Routing Problems in Real-time. arXiv:2312.08224 [cs.AI] https://arxiv.org/abs/2312.08224

[31] Changliang Zhou, Xi Lin, Zhenkun Wang, Xialiang Tong, Mingxuan Yuan, and Qingfu Zhang. 2024. Instance conditioned adaptation for large-scale generalization of neural combinatorial optimization. *arXiv:2405.01906, 2024* (2024).


## A Notations

In order to improve the clarity of reading, we list all notations and their descriptions used in this paper, as shown in Tab. 4.

## B Detailed results on CVRPLib

We summarized the performance of the baseline models on the tested CVRPLib dataset in Tab. 5, which includes the path lengths obtained by the baseline models as well as RANS on the specific CVRPLib instances.

**Table 4: The notations used in this paper.**

| Notation | Description | Notations | Description |
|---|---|---|---|
| $M$ | The number of vehicles | $N$ | The number of vehicles |
| $\rho_i$ | The capacity of each vehicle $i$ | $\chi_i$ | The speed of each vehicle $i$ |
| $b_j$ | The x-coordinate of each customer or depot node $j$ | $c_j$ | The y-coordinate of each customer or depot node $j$ |
| $\delta_j$ | The demand of each customer or depot node $j$ | $\mathcal{T}$ | The solution of an HCVRP instance |
| $\mathcal{T}^*$ | The optimal solution of an HCVRP instance | $\tau^i$ | The path of the each vehicle $i$ in $\mathcal{T}$ |
| $\mathcal{M}$ | The Markov Decision Process (MDP) | $\mathcal{S}$ | The state space in MDP |
| $\mathcal{A}$ | The action space in MDP | $\mathcal{F}$ | The transition function in MDP |
| $R$ | The reward function in MDP | $x_j$ | the static state of the node $j$ |
| $y_i^t$ | the state of the vehicle $i$ at step $t$ | $\hat{\rho}_i^t$ | The accumulated usage capacity of vehicle $i$ at step $t$ |
| $\Upsilon_i^t$ | The accumulated time of vehicle $i$ at step $t$ | $g_j^t$ | Whether node $j$ has been served at step $t$ |
| $\ell_i^t$ | The node where the vehicle $i$ is located at step $t$ | $s^t$ | The state at step $t$ |
| $a^t$ | The action at step $t$ | $\pi_\theta$ | The policy with trainable parameters $\theta$ |
| $f^n$ | The node encoder | $\theta_1$ | The trainable parameters in $f^n$ |
| $f^v$ | The vehicle encoder | $\theta_2$ | The trainable parameters in $f^v$ |
| $f^d$ | The action decoder | $\beta^t$ | The attention score in $f^d$ at step $t$ |
| $P^t$ | The action probability at step $t$ | $\mathbf{x}$ | The node embeddings |
| $\mathbf{y}^t$ | The vehicle embeddings at step $t$ | $\mathbf{x}_j^0$ | The raw features of each node $j$ |
| $\mathbf{x}_j^1$ | The transformed features of each node $j$ | $\mathbf{H}^l$ | The output of the l-th node encoder layer |
| $DT$ | The depot token in the node encoder | $W$ | Some trainable parameter matrices |
| $O$ | The output of the Rank-Augmented Module (RA) | $Q$ | The Query in each attention module |
| $K$ | The Key in each attention module | $V$ | The Value in each attention module |
| $M_c$ | The coefficient matrix with higher rank upper bound | $A$ | The adjacency matrix |
| $\hat{A}$ | The adjacency matrix with self-connections | $D$ | The degree matrix of $\hat{A}$ |
| $I_N$ | Identity matrix | $\mathbf{y}_i^{t,0}$ | The raw features of each vehicle $i$ |
| $\mathbf{y}_i^{t,1}$ | The first Query of the Dual Cross-Attention Module | $\mathbf{y}_i^{t,2}$ | The second Query of the Dual Cross-Attention Module |
| $\mathbf{h}^1$ | The vehicle feature associated with the navigation task | $\mathbf{h}^2$ | The vehicle feature associated with the partition task |
| $\mathbf{e}^t$ | The mask embeddings | $\mathbf{x}^{mask}$ | The positional encoding |
| $d$ | The main dimension in node or vehicle encoder | $d'$ | The head dimension in various attention modules |
| $\mathcal{L}_D$ | The disagreement regularization loss | $\mathcal{L}$ | The overall loss |
| $\alpha$ | The weight of the $\mathcal{L}_D$ | $B$ | The size of each batch |
| $U$ | The number of symmetric solutions for augmentation | $G$ | The advantage function |

**Table 5: Performance comparison in real-world instances in CVRPLib.**

| Instance | opt-cost | AM-cost | ET-cost | 2D-Ptr-cost | RANS-cost |
|---|---|---|---|---|---|
| Golden_1 | 5623.47 | 10554.01 | 15132.70 | 6990.50 | 5908.51 |
| Golden_13 | 857.19 | 1365.57 | 2649.41 | 1218.06 | 935.28 |
| Golden_14 | 1080.55 | 1895.22 | 3317.41 | 1557.80 | 1211.11 |
| Golden_15 | 1337.27 | 4490.31 | 4285.23 | 2064.61 | 1502.46 |
| Golden_16 | 1611.28 | 4253.78 | 5584.24 | 2491.18 | 1823.22 |
| Golden_17 | 707.76 | 1964.01 | 1668.27 | 924.09 | 786.78 |
| Golden_18 | 995.13 | 3352.60 | 2493.15 | 1372.16 | 1136.69 |
| Golden_2 | 8404.61 | 17912.23 | 24178.14 | 11975.80 | 9221.66 |
| Golden_9 | 579.70 | 909.48 | 1505.48 | 740.43 | 649.28 |
| Li_22 | 14499.04 | 37404.00 | 50141.15 | 22784.60 | 15975.41 |
| Li_25 | 16665.70 | 44377.37 | 57956.21 | 27551.69 | 18425.06 |
| Loggi-n401-k23 | 336903.00 | 501331.00 | 592738.00 | 398015.00 | 366623.00 |
| M-n101-k10 | 820.00 | 1218.66 | 2065.35 | 1044.12 | 833.43 |
| M-n121-k7 | 1034.00 | 1417.51 | 2687.78 | 1259.15 | 1109.38 |
| M-n151-k12 | 1015.00 | 1747.26 | 2923.78 | 1389.14 | 1134.50 |
| M-n200-k17 | 1275.00 | 2214.33 | 3672.55 | 1771.51 | 1482.32 |
| X-n101-k25 | 27591.00 | 37783.17 | 39624.88 | 32428.53 | 28529.19 |
| X-n106-k14 | 26362.00 | 30329.21 | 34524.04 | 27049.43 | 25974.73 |
| X-n110-k13 | 14971.00 | 20411.71 | 33036.62 | 17712.05 | 15769.48 |
| X-n115-k10 | 12747.00 | 20591.09 | 35551.54 | 16686.09 | 14145.98 |
| X-n120-k6 | 13332.00 | 16880.24 | 33789.40 | 16102.97 | 13789.80 |
| X-n125-k30 | 55539.00 | 71846.55 | 69797.12 | 64277.09 | 57916.97 |
| X-n129-k18 | 28940.00 | 47667.20 | 45940.26 | 33096.37 | 30111.43 |
| X-n134-k13 | 10916.00 | 17452.18 | 21471.02 | 12789.36 | 12059.43 |
| X-n139-k10 | 13590.00 | 19800.79 | 38814.00 | 17622.89 | 14971.77 |
| X-n143-k7 | 15700.00 | 28011.06 | 42653.98 | 20448.85 | 17641.76 |
| X-n148-k46 | 43448.00 | 64641.73 | 63419.97 | 51999.02 | 45422.63 |
| X-n153-k22 | 21220.00 | 29983.53 | 36745.38 | 25469.77 | 25021.13 |
| X-n157-k13 | 16876.00 | 21840.22 | 31670.91 | 18897.71 | 18389.76 |
| X-n162-k11 | 14138.00 | 19934.99 | 38669.81 | 18057.24 | 15663.43 |
| X-n167-k10 | 20557.00 | 30791.26 | 42875.04 | 25559.48 | 22649.79 |
| X-n172-k51 | 45607.00 | 66306.33 | 70097.11 | 59437.44 | 47879.11 |
| X-n176-k26 | 47812.00 | 70442.66 | 82322.35 | 59876.17 | 53822.82 |
| X-n181-k23 | 25569.00 | 31058.15 | 39411.29 | 29043.31 | 26638.01 |
| X-n186-k15 | 24145.00 | 35827.44 | 44765.60 | 29964.49 | 26179.24 |
| X-n190-k8 | 16980.00 | 27596.43 | 33752.89 | 19770.77 | 18448.51 |
| X-n195-k51 | 44225.00 | 84226.31 | 73235.05 | 56479.26 | 46592.54 |
| X-n200-k36 | 58578.00 | 76700.62 | 74431.86 | 66947.10 | 61334.13 |
| X-n204-k19 | 19565.00 | 28797.66 | 48560.78 | 23897.53 | 21233.09 |
| X-n209-k16 | 30656.00 | 47404.59 | 65019.42 | 38449.21 | 33307.86 |
| X-n223-k34 | 40437.00 | 80501.10 | 76648.21 | 49282.59 | 43260.58 |
| X-n228-k23 | 25742.00 | 43555.78 | 54280.82 | 32699.03 | 29903.65 |
| X-n233-k16 | 19230.00 | 39376.89 | 57475.06 | 26294.94 | 21096.88 |
| X-n237-k14 | 27042.00 | 36579.07 | 59335.53 | 34720.49 | 31433.50 |
| X-n242-k48 | 82751.00 | 221394.79 | 131624.81 | 107408.91 | 86966.12 |
| X-n247-k50 | 37274.00 | 54516.29 | 60764.61 | 47707.84 | 42930.44 |
| X-n251-k28 | 38684.00 | 56197.04 | 63664.27 | 46880.58 | 41486.35 |
| X-n256-k16 | 18839.00 | 26854.20 | 46346.95 | 24934.93 | 20972.93 |
| X-n261-k13 | 26558.00 | 48367.80 | 64651.76 | 36197.47 | 30368.71 |
| X-n266-k58 | 75478.00 | 175446.50 | 115052.65 | 89083.66 | 79574.82 |
| X-n270-k35 | 35291.00 | 55327.93 | 81859.02 | 43996.57 | 37668.24 |